

Plotly.js crosshair tracking CPU issues: the hover system never stops

The root cause is likely Plotly's internal hover detection algorithm, which performs $O(n)$ point-finding calculations on every mousemove event regardless of whether `hovermode` is disabled or `plotly_beforehover` returns false. This processing continues because the event only prevents tooltip display, not the underlying computation. Combined with potential layout-forcing DOM reads like `event.offsetX` or `getBoundingClientRect()` inside the animation loop, these factors create sustained CPU load during mouse tracking. [Chrome Developers](#)

Plotly's hover system runs even when "disabled"

GitHub issues #3826, #5641, and #5522 document a critical limitation: **Plotly's hover detection loops over every data point on each mousemove** to find the closest point to the cursor. The source code in `src/components/fx/helpers.js` contains an explicit comment: *"apply the distance function to each data point - this is the longest loop."* [GitHub](#)

The `plotly_beforehover` returning false only prevents the tooltip from being shown—it **does NOT prevent the underlying point detection calculations**. Issue #5522 confirms: *"Using plotly_beforehover to prevent the default tooltip requires that you implement your own unhover event+processing somehow."* [GitHub](#) The coordinate calculations in `gd._fullLayout.xaxis.l2p()` and the point-finding algorithm in `src/components/fx/hover.js` still execute.

Additionally, Plotly's `dragElement` system attaches document-level mousemove listeners that fire even when hovering is "disabled," triggering `gd._dragged` state checks and potential `_rehover` invocations. The `_rehover` mechanism, designed for streaming data scenarios, can create cascade effects when combined with plot modifications. [GitHub](#)

The offsetX/offsetY layout trap

A likely hidden culprit is reading `event.offsetX` or `event.offsetY` in the mousemove handler. **These properties force synchronous layout** because the browser must calculate the element's position to compute the offset. Paul Irish's authoritative list of layout-forcing properties explicitly includes `mouseEvt.layerX`, `mouseEvt.layerY`, `mouseEvt.offsetX`, and `mouseEvt.offsetY`. [GitHub](#)

The fix is straightforward: use `event.clientX` and `event.clientY` instead, computing offsets from a cached `getBoundingClientRect()` that updates only on resize:

javascript

```
let cachedRect = chartElement.getBoundingClientRect();
window.addEventListener('resize', () => {
  cachedRect = chartElement.getBoundingClientRect();
});

function handleMouseMove(e) {
  // Good: no layout forced
  const x = e.clientX - cachedRect.left;
  const y = e.clientY - cachedRect.top;
}
```

Similarly, calling `getBoundingClientRect()` inside the animation loop forces layout if the DOM has been modified since the last read. [Webperf](#) [Nolan Lawson](#) The user stated they ruled this out, but verify it isn't being called indirectly through `getPlotCoordinates()` or any helper functions.

staticPlot configuration eliminates internal event binding

The most effective solution is using `staticPlot: true` in the Plotly config. This completely disables Plotly's internal event binding and eliminates the hover detection overhead. [Plotly](#) Source code analysis shows `staticPlot` forces `editable`, `autosizable`, `scrollZoom`, and `doubleClick` all to false, [GitHub](#) and critically **omits invisible elements used only for capturing interactions.** [GitHub](#)

javascript

```
Plotly.newPlot('chart', data, layout, { staticPlot: true });

// Add custom overlay for all mouse interaction
const overlay = document.createElement('div');
overlay.style.cssText = 'position:absolute;inset:0;pointer-events:auto;';
overlay.addEventListener('mousemove', handleCrosshair);
document.getElementById('chart').appendChild(overlay);
```

If full interactivity is needed (zoom, pan), an alternative approach targets Plotly's internal drag layer:

javascript

```
const dragLayer = document.querySelector('#chart .nsewdrag');
if (dragLayer) dragLayer.style.pointerEvents = 'none';
```

This prevents Plotly from receiving mouse events while your overlay handles them, though it disables built-in interactions.

CSS will-change creates compositing layer overhead

The `will-change: transform` property on crosshair lines creates separate GPU compositing layers. Each layer requires texture memory (approximately width × height × 4 bytes), CPU-to-GPU bandwidth for uploads, and browser management overhead. [Gitbook](#) Having this on two crosshair lines is fine, but applying it to containers or many elements causes "layer explosion."

Key issue: if the crosshair overlay shares a compositing layer with the Plotly chart, transform updates can trigger repaints in the underlying canvas/SVG. Ensure proper layer separation:

CSS

```
.chart-container {  
  position: relative;  
  isolation: isolate; /* Creates stacking context, contains layers */  
}  
  
.crosshair-overlay {  
  position: absolute;  
  inset: 0;  
  pointer-events: none;  
  contain: layout; /* Isolates layout calculations */  
}  
  
.crosshair-line {  
  position: absolute;  
  will-change: transform; /* Own compositing layer */  
  pointer-events: none;  
}
```

The `contain: layout` property provides minimal benefit for transform-only updates but helps isolate the overlay from chart DOM interactions. [Smashing Magazine](#) Use Chrome DevTools Layers panel (More Tools → Layers) to verify crosshairs have their own layers and the chart is on a separate layer.

RAF accumulation without guard flags

If `requestAnimationFrame()` is called inside the mousemove handler without a guard, callbacks accumulate because mousemove fires faster than the frame rate [Nolan Lawson](#) (mice report at 125–1000Hz). The correct pattern:

javascript

```
let rafId = null;
let pendingX = 0, pendingY = 0;

function onMouseMove(e) {
  pendingX = e.clientX - cachedRect.left;
  pendingY = e.clientY - cachedRect.top;

  if (!rafId) {
    rafId = requestAnimationFrame(() => {
      verticalLine.style.transform = `translateX(${pendingX}px)`;
      horizontalLine.style.transform = `translateY(${pendingY}px)`;
      rafId = null;
    });
  }
}
```

Also check for memory leaks: RAF callbacks not cancelled on component unmount, closures holding references to DOM elements, or dispose handlers that never clear. [GitHub](#)

Chrome DevTools diagnostic workflow

Performance Monitor panel (More Tools → Performance monitor) provides real-time metrics during Shift+mousemove:

- **CPU usage:** sustained high percentage indicates the bottleneck [LogRocket](#)
- **Layouts/sec:** should be near zero during smooth animation—spikes indicate layout thrashing [Chrome Developers](#)
- **Style recalcs/sec:** high values suggest forced style recalculation
- **JS heap size:** growth over time indicates memory leaks [LogRocket](#)

Performance tab recording reveals the bottleneck type: [DebugBear](#)

Pattern	Indicates
Tall yellow stacks	Heavy JavaScript (Plotly's hover algorithm) JiaYihu
Purple "Recalculate Style" blocks with red triangles	Forced synchronous layout
Green blocks >16ms	Paint bottleneck
Repeated "Update Layer Tree" + "Composite Layers"	Layer thrashing

Click on purple blocks to see the exact line forcing layout ("Recalculation Forced") and what invalidated it ("First Invalidated"). [DebugBear](#)

Binary search isolation: Comment out the entire crosshair update function—if CPU drops, uncomment half and repeat until the specific operation is found. [Vladimirzdrazil](#)

Cursor style changes invalidate entire page layout

If Shift+mousemove changes the cursor style (`document.body.style.cursor = 'crosshair'`), this **invalidates layout for the entire page**. GitHub comments from browser engineers confirm cursor changes trigger expensive recalculations. Either avoid changing the cursor during tracking or set it once when Shift is pressed rather than on every mousemove.

SVG vs WebGL provides no event handling benefit

WebGL renderers (`scattergl`, `gl2d`) accelerate **rendering** but not **event handling**. [Plotly](#) The hover point-detection algorithm runs in JavaScript regardless of renderer type. GitHub issue #5927 reports that even scattergl with 100k points becomes laggy during mouse interactions because the bottleneck is JavaScript point-finding, not rendering. The only benefit comes from reduced paint time, not from event processing overhead.

Conclusion

The most probable causes are: **(1)** Plotly's hover detection algorithm running $O(n)$ calculations despite disabled hovermode, **(2)** layout-forcing reads like `event.offsetX` or `getBoundingClientRect()` in the animation loop, [Screaming Frog](#) **(3)** RAF callbacks accumulating without proper guards, and **(4)** cursor style changes on Shift invalidating page layout.

The definitive fix is `staticPlot: true` with a custom event overlay that bypasses Plotly's event system entirely. For preserving interactivity, disable pointer events on `.nsewdrag`, use `clientX/clientY` with cached rect, [Afafterweb](#) guard RAF calls, and verify layer separation in DevTools. The diagnostic workflow should start with Performance Monitor for real-time CPU and layouts/sec, then move to Performance tab recording to identify yellow (JS) vs purple (layout) bottlenecks with exact call stacks. [LogRocket](#)